Atividade Prática — Aula 02

API Design, DTOs (Data Transfer Objects) & Jakarta Bean Validation

1. Guia Teórico & Boas Práticas da Aula 02

Este guia resume os conceitos fundamentais apresentados na Aula 02 e adiciona **dicas práticas de arquitetura e armadilhas comuns** para apoiar a realização das atividades no IntelliJ IDEA e no cliente HTTP Bruno.

1.1. Semântica REST & Códigos de Resposta HTTP

Uma REST API corporativa profissional deve ser previsível, utilizando os verbos HTTP adequados e retornando códigos de status que reflitam fielmente o resultado do processamento:

Método	Semântica REST	Idempotente?	Status Code Esperado
GET	Consulta de recurso(s). Nunca altera estado.	Sim	200 OK / 404 Not Found
POST	Criação de um novo recurso no sistema.	Não	201 Created (+ Location) / 400 Bad Request
PUT	Substituição COMPLETA do recurso existente.	Sim	200 OK / 404 Not Found / 400 Bad Request
PATCH	Atualização PARCIAL (cirúrgica) de atributos.	Não*	200 OK / 404 Not Found / 400 Bad Request
DELETE	Remoção de um recurso pelo seu ID.	Sim	204 No Content / 404 Not Found

> **DICA PRÁTICA: PUT vs PATCH & Cabeçalho Location**
>
> • DIFErença CRUCIAL (PUT vs PATCH): O PUT exige o envio de TODOS os campos do recurso. Se enviar apenas o atributo alterado num PUT, os outros campos serão sobrescritos com null! Use PATCH para alterações parciais isoladas (ex: alterar apenas o rating do filme).
>
> • CABEÇALHO LOCATION NO POST: Ao criar um recurso com 201 Created, é uma boa prática corporativa devolver o cabeçalho 'Location: http://localhost:8080/api/movies/4' indicando onde o novo recurso pode ser consultado.

1.2. Desacoplamento de Arquitetura via DTOs (Records Java 21)

Regra de Ouro da Arquitetura Spring: Nunca exponha as suas Entidades de Domínio (@Entity / Model) diretamente nos parâmetros dos Controllers ou nos retornos da API. Utilize **DTOs (Data Transfer Objects)** para separar o contrato público externo das regras de negócio internas.

- **Vantagens dos Records (Java 21):** Classes imutáveis, sintaxe extremamente concisa, atributos 'final', construtor compacto e geração automática de getters, equals(), hashCode() e toString().
- **Separação Entrada vs Saída:** Use DTOs distintos para criação (ex: MovieRequest) e para resposta da API (ex: MovieResponse). Isso previne que o cliente tente injetar IDs manualmente ou altere campos calculados/internos.

> ⚠ **DICA DE ARQUITETURA: Jackson & Mapeamento de Atributos**
>
> • ARMADILHA DO JACKSON (Mapeamento JSON): Se o cliente enviar uma chave JSON como 'titulo' e o seu Record Java tiver o campo 'title', o Jackson não conseguirá mapear o valor e o campo ficará null silenciosamente!
>
> • SOLUÇÃO: Caso o contrato externo difira do nome do campo em Java, utilize a anotação @JsonProperty("titulo") ou @JsonAlias("titulo") no parâmetro do Record.

1.3. Validação Declarativa com Jakarta Bean Validation

Em vez de escrever múltiplos bloques `if (request.title() == null || request.title().isBlank())` dentro do Controller, utilize as anotações declarativas do pacote `jakarta.validation.constraints` diretamente no DTO de entrada:

Anotação	Tipos Aplicáveis	Comportamento de Validação
`@NotBlank`	String	Não permite null, texto vazio ("") nem composto apenas por espaços (" ").
`@NotNull`	Qualquer Tipo	Não permite valor null. Exige que a propriedade seja informada no JSON.
`@Size(min, max)`	String / Collection	Valida a quantidade de caracteres da String ou elementos da coleção.
`@Min / @Max`	Inteiros / Long	Valida os limites numéricos mínimos e máximos permitidos.
`@DecimalMin / Max`	Double / BigDecimal	Valida limites decimais (ex: @DecimalMin(value="0.0") e @DecimalMax(value="10.0")).

> **ARMADILHA CLÁSSICA: O papel do @Valid**
> • O SEGREDO DO @VALID: Adicionar @NotBlank no Record NÃO VALIDA NADA por si só!
> • Para que o Spring ative o inspetor de validação na entrada do endpoint, é OBRIGATÓRIO incluir a anotação @Valid antes de @RequestBody no método do Controller:
> public ResponseEntity<MovieResponse> create(@Valid @RequestBody MovieRequest request)
> • Se esquecer o @Valid, o Spring aceitará títulos em branco e retornará 200/201 normalmente!

2. Baseline Git & Estrutura Oficial de Pacotes

Caso necessite de alinhar a sua máquina local com o estado oficial aprovado no final da Aula 01, execute no Terminal do IntelliJ IDEA:

Terminal do IntelliJ (Sincronização com Aula 01)

```
git switch main
git fetch origin --tags
git reset --hard checkpoint/aula-01-rest
```

Estrutura oficial de diretórios e pacotes para a Aula 02 sob **pt.rumos.mymovies:**

Árvore de Pacotes do Projeto MyMovies

```
src/main/java/pt/rumos/mymovies/
├── MyMoviesApplication.java
├── model/
│   └── Movie.java                  // Entidade de domínio (id, title, director, year,
rating)
├── dto/
│   ├── MovieRequest.java           // DTO de entrada POST/PUT (@NotBlank, @Size, @Min)
│   ├── MovieResponse.java          // DTO de saída da API
│   └── controller/
    └── MovieController.java        // Controlador REST com endpoints CRUD e @Valid
```

3. Roteiro Prático Guiado

Siga os passos abaixo para refatorar a API MyMovies, aplicando DTOs, regras de validação e verbos HTTP corretos.

Adicionar no pom.xml do projeto a seguinte dependência:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Passo 1: Criar os DTOs de Contrato Público (MovieRequest e MovieResponse)

1.1. No pacote `pt.rumos.mymovies.dto`, crie o record `MovieRequest` para representar os dados de entrada na criação de um filme com as seguintes validações:

- **title:** String obrigatória (não em branco com @NotBlank) e máximo 120 caracteres (@Size(max = 120)).
- **director:** String obrigatória (não em branco com @NotBlank).
- **year:** Integer obrigatório (@NotNull) e no mínimo 1888 (@Min(1888)).

1.2. No mesmo pacote, crie o record `MovieResponse` para ser devolvido nas consultas. Atributos: `Long id, String title, String director, Integer year`. Adicione um método estático `fromDomain(Movie movie)` que converte um objeto de domínio para DTO.

Passo 2: Implementar os Endpoints no MovieController

Refatore a classe `MovieController` no pacote `pt.rumos.mymovies.controller` implementando os seguintes métodos:

- **GET /api/movies (Listar Filmes):** Retorna ResponseEntity<List<MovieResponse>> mapeando todos os filmes da coleção em memória para DTOs com status 200 OK.
- **POST /api/movies (Criar Filme):** Recebe @Valid @RequestBody MovieRequest request. Deve gerar um novo ID, criar a instância de Movie, adicionar à lista e retornar status **201 Created** juntamente com o cabeçalho Location (usando ServletUriComponentsBuilder) e o MovieResponse no corpo.
- **PUT /api/movies/{id} (Atualização Completa):** Recebe @PathVariable Long id e @Valid @RequestBody MovieRequest request. Procura o filme pelo ID. Se não encontrar, devolve **404 Not Found**. Se encontrar, substitui os dados (title, director, year) mantendo o ID, atualiza a lista e devolve **200 OK** com o MovieResponse atualizado.
- **DELETE /api/movies/{id} (Remover Filme):** Recebe @PathVariable Long id. Tenta remover o filme da coleção pelo ID (ex: movies.removeIf(m -> m.id().equals(id))). Se for removido, devolve **204 No Content** (sem corpo). Caso o ID não exista, devolve **404 Not Found**.

Esqueleto de Apoio (MovieController.java)

Como vamos simular inclusão, atualização e remoção de elementos no catálogo e, como ainda os dados não estão em base de dados, precisamos tornar a lista mutável encapsulando a lista original em um *ArrayList*

```
// GUIA DE IMPLEMENTAÇÃO (ESQUELETO DO CONTROLLER COM TODOs)
@RestController
@RequestMapping("/api/movies")
public class MovieController {

    private final List<Movie> movies = new ArrayList<>(List.of(
        new Movie(1L, "Inception", "Christopher Nolan", 2010),
        new Movie(2L, "The Matrix", "Lana & Lilly Wachowski", 1999)
        //outros filmes da lista
    ));

    // TODO: Implementar GET /api/movies -> Já Implementado
    // TODO: Implementar POST /api/movies -> Com @Valid @RequestBody e 201 Created
    // TODO: Implementar PUT /api/movies/{id} -> Com @Valid, 200 OK ou 404 Not Found
    // TODO: Implementar DELETE /api/movies/{id} -> Retornar 204 No Content ou 404 Not
Found
}
```

Passo 3: Validar os Endpoints no Cliente Bruno

Abra o cliente HTTP Bruno e crie/execute as requisições para validar os comportamentos da sua API:

- **Teste GET:** Chame GET http://localhost:8080/api/movies e confirme o retorno 200 OK com o JSON dos filmes.
- **Teste POST Válido:** Envie um JSON válido e verifique se o status retornado é 201 Created e se o cabeçalho Location está preenchido.
- **Teste POST Inválido (Bean Validation):** Envie um JSON com título em branco (" ") ou ano 1500. Verifique se o Spring intercepta e responde automaticamente com **400 Bad Request**.
- **Teste PUT & DELETE:** Atualize o filme 1 via PUT e em seguida remova o filme 2 via DELETE, confirmando os retornos 200 OK, 204 No Content e 404 Not Found para IDs inexistentes.

4. Checklist de Conclusão (Definition of Done)

Antes de finalizar a atividade, certifique-se de que cumpre todos os critérios de aceite:

- **[ ]** A aplicação compila e inicia sem erros no IntelliJ IDEA.
- **[ ]** O GET /api/movies retorna a lista de MovieResponse.
- **[ ]** O POST /api/movies cria o filme, retorna 201 Created com header Location e invalida payloads incorretos com 400 Bad Request.
- **[ ]** O PUT e o DELETE tratam IDs inexistentes retornando 404 Not Found e 204 No Content para deleção efetuada.